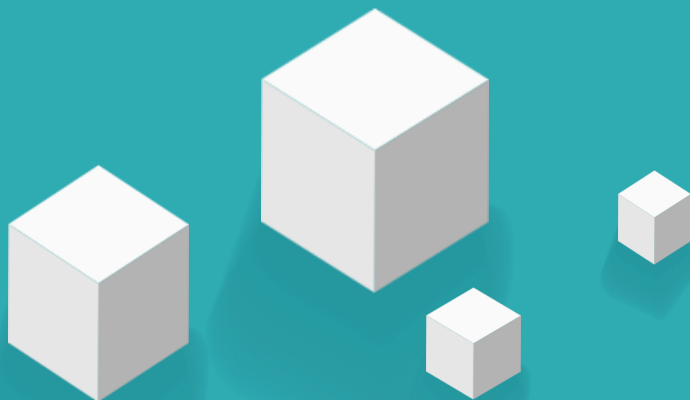
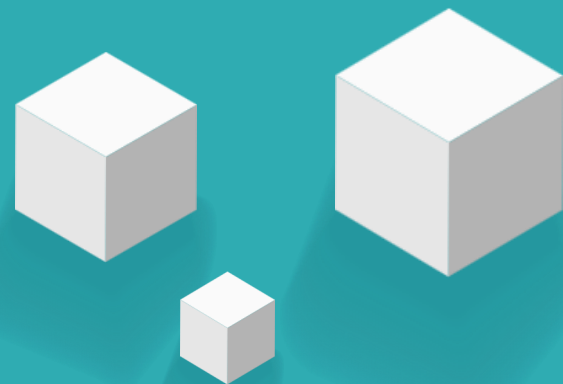


vue.js
AUG 12

第七章 单文件组件

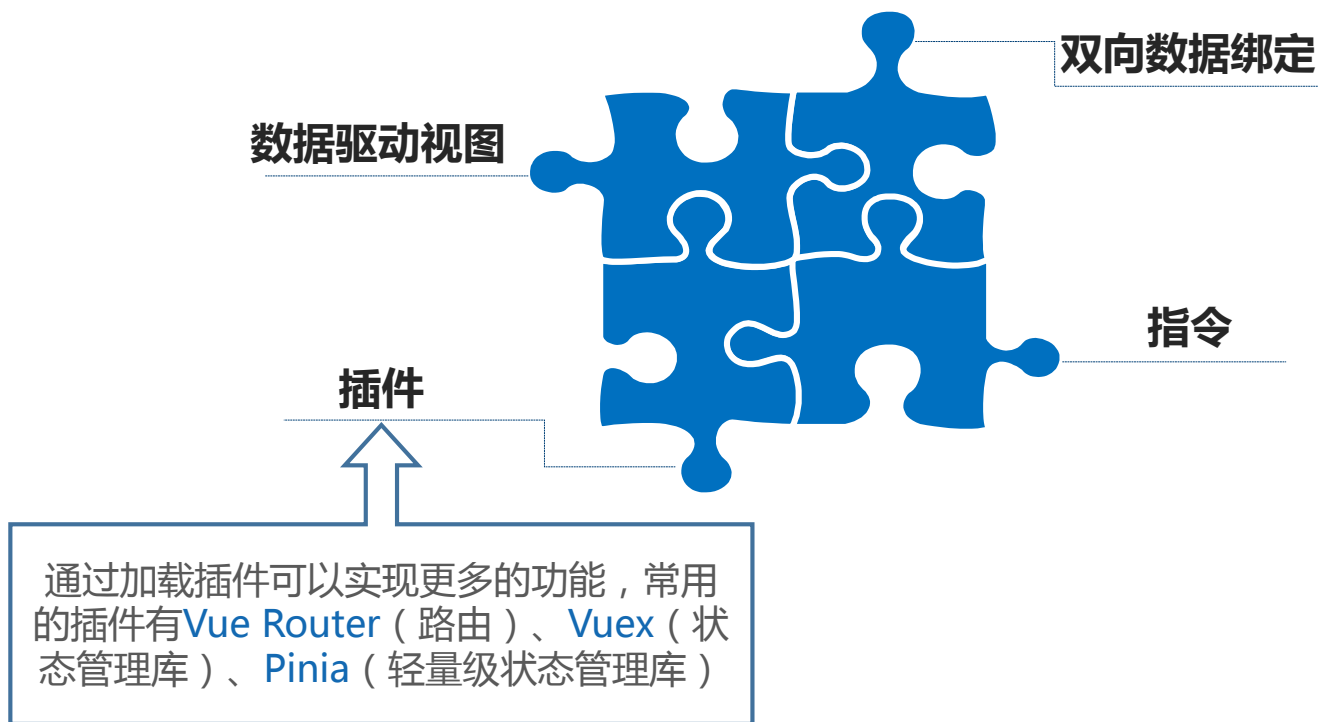


»» 第7章 单文件组件

- 开发环境
- 单文件组件
- 数据绑定

»» 7.1 开发环境

7.1.1 Vue的特性



Vue 3和Vue 2的**使用方式**没有太大的差异，但Vue 3的**底层代码**发生了很大变化，包括渲染、数据监听、双向绑定、生命周期等。

»» 7.1 开发环境

7.1.2 常见的包管理工具

- 在Vue项目开发中，经常需要通过各种第三方的包（package，将一系列模块化的代码打包起来便于使用）来扩展项目的功能。项目中所用到的包称为项目的依赖（dependency）。
- 包管理工具：管理第三方包，让开发人员轻松地下载、升级、卸载包。

常见的包管理工具



npm

npm是Node.js默认的包管理工具，它可以安装、共享、分发代码，还可以管理项目的依赖关系。



yarn

yarn是Node.js的包管理工具，它是一个高效、安全和可靠的包管理工具，yarn能够提高包的安装效率，节约安装时间。

» 7.1 开发环境

7.1.3 包管理工具npm

在安装Node.js时会[自动安装相应版本的npm](#)，不需要单独安装，使用“`npm -v`”命令可以查看npm的版本。



```
C:\Users\itcast>npm -v
8.15.0
C:\Users\itcast>_
```

使用npm包管理工具可以解决以下场景的[需求](#)。

- ① 从npm服务器下载[别人编写的第三方包](#)到本地使用。
- ② 从npm服务器下载并安装[别人编写的命令行程程序](#)到本地来使用。
- ③ 将[自己编写的包或命令行程程序](#)上传到npm服务器供别人使用。

»» 7.1 开发环境

7.1.3 包管理工具npm

npm提供了快速操作包的命令，只需要执行简单的命令就可以很方便地对第三方包进行管理。

npm中常用的命令：

- **npm install 包名**：可简写为“npm i 包名”，为项目安装指定名称的包。
 - -g选项：把包安装为全局包，否则只安装到本项目中。
 - 省略包名，npm会根据当前目录下的package.json文件中保存的依赖信息为项目安装所有的包。
- **npm uninstall 包名**：卸载指定名称的包。
- **npm update 包名**：更新指定名称的包。

»» 7.1 开发环境

7.1.4. 使用vite创建Vue3项目

- Vite：是一个轻量级、运行速度快的前端构建工具，它支持模块热替换（Hot Module Replacement，HMR），可以即时、准确地更新模块，当代码修改时无须重新加载页面或清除应用程序状态。Vite默认安装的插件比较少，随着开发过程中依赖的增多，需要额外进行配置。
- Vite不需要以命令的方式安装和卸载，只要安装了npm或yarn，就可使用Vite的相关命令创建项目。Vite会作为项目的开发依赖保存在项目的node_modules目录中。

» 7.1 开发环境

7.1.4 通过模板创建Vue 3项目

- 通过模板自动创建项目的方式相对简单，它通过附加的命令行选项直接指定项目名称和模板，省去了填写项目名称、选择框架和变体等环节。
- 在VSCode终端面板中输入`npm init vue@latest`命令，就会自动安装并执行create-vue（Vue 3专门的脚手架工具）
- 例如：用npm创建一个基于Vite+Vue3模板且项目名称为helloworld的项目。



```
问题  输出  调试控制台  终端

PS D:\code\helloworld> npm init vue@latest

Vue.js - The Progressive JavaScript Framework

? Project name: » vue-project helloWorld
```

»» 7.1 开发环境

7.1.4 通过模板创建Vue 3项目

- create-vue脚手架工具自动执行创建Vue 3项目期间，有可选功能提示可自行选择安装。

```
PS D:\code\helloworld> npm init vue@latest
```

Vue.js - The Progressive JavaScript Framework

```
✓ Project name: ... helloworld
✓ Add TypeScript? ... No / Yes
✓ Add JSX Support? ... No / Yes
✓ Add Vue Router for Single Page Application development? ... No / Yes
✓ Add Pinia for state management? ... No / Yes
✓ Add Vitest for Unit Testing? ... No / Yes
✓ Add an End-to-End Testing Solution? » No
✓ Add ESLint for code quality? ... No / Yes
```

路由组件

Pinia状态管理库

Vitest单元测试

端到端测试方法

代码检测工具

Scaffolding project in D:\code\helloworld\helloworld...

Done. Now run:

```
cd helloworld
npm install
npm run dev
```

○ PS D:\code\helloworld> █

»» 7.1 开发环境

7.1.4 通过模板创建Vue 3项目

在项目创建完成后，给出了一些提示命令，需要执行这些命令才能完成项目启动，具体命令如下。

```
cd <project-name>    # 切换到项目目录
npm install            # 安装项目的全部依赖
npm run dev            # 启动服务
```

项目启动后，会默认开启一个本地服务。

```
VITE v4.1.4 ready in 441 ms
→ Local: http://127.0.0.1: xxxx/
```

» 7.1 开发环境

7.1.5 Vue 3项目的目录结构

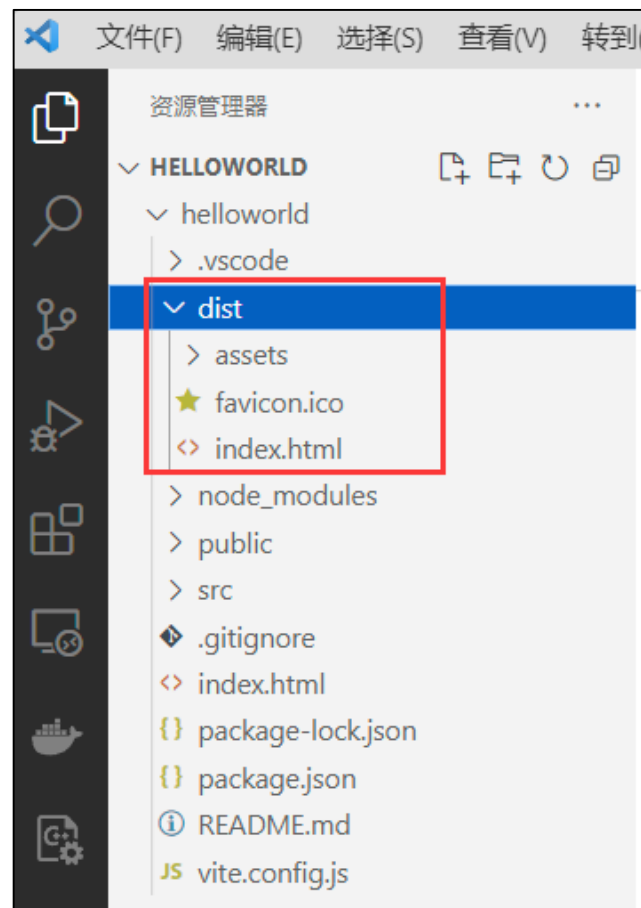
- npm run build命令后，在项目目录中会出现dist目录，该目录中保存的是项目构建后的文件。（[生成可用于生产环境的优化代码。](#)）
- Vue 3项目创建成功后，构建HelloWorld项目，可以看到一个默认生成的项目目录结构。

```
问题 输出 调试控制台 终端

PS D:\code\helloworld\helloworld> npm run build

> helloworld@0.0.0 build
> vite build

vite v4.4.7 building for production...
✓ 42 modules transformed.
dist/assets/logo-277e0e97.svg      0.28 kB | gzip: 0.20 kB
dist/index.html                    0.42 kB | gzip: 0.28 kB
dist/assets/AboutView-4d995ba2.css 0.09 kB | gzip: 0.10 kB
dist/assets/index-efcb3133.css     4.20 kB | gzip: 1.30 kB
dist/assets/AboutView-923dc868.js   0.23 kB | gzip: 0.20 kB
dist/assets/index-3541e00c.js      84.97 kB | gzip: 33.45 kB
✓ built in 689ms
○ PS D:\code\helloworld\helloworld>
```



»» 7.1 开发环境

整体项目结构

HelloWorld	.vscode	存放VS Code编辑器的相关配置
	dist	项目构建文件目录
	node_modules	项目依赖包存放目录，npm或yarn安装的库和框架存放这里
	public	存放不可编译的静态资源文件，进行项目构建时，该目录下的文件会被复制到dist目录（使用绝对路径访问）。
	src	项目的源文件，编写的核心代码都存放于此
	.gitignore	向Git仓库上传代码时需要忽略的文件列表
	index.html	默认的主渲染页面文件，同时也是页面的入口文件
	package-lock.json	锁定依赖包安装版本号文件，保证大家使用同一版本号依赖包
	package.json	node.js项目配置文件和包管理文件
	readme	项目说明文档
	Vite.config.js	Vite相关配置文件

7.1 开发环境

src项目源文件目录

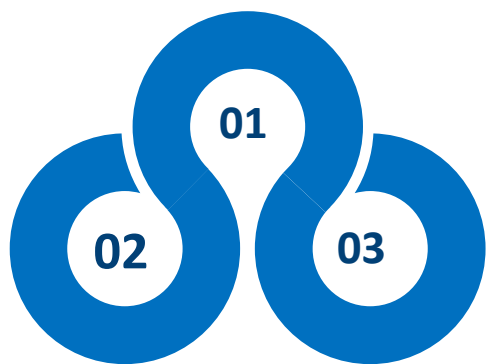
- 源代码目录，保存开发人员编写的项目源代码。



»» 7.1 开发环境

7.1.6 Vue 3项目的运行过程

使用Vite构建Vue 3项目后，当执行`npm run dev`命令启动服务时，项目就会运行起来，该项目会通过`src/main.js`文件将`src/App.vue`组件渲染到`index.html`文件的指定区域。



1

`src/App.vue`文件

项目的**根组件**。在根组件中可以引用其他组件，从而显示其他组件的内容。（Vue 3项目是由各种组件组成的）

2

`index.html`文件

页面的入口文件，该文件中预留了用于被`src/main.js`文件中的Vue应用实例所控制的区域。

3

`src/main.js`文件

项目的入口文件，该文件创建了Vue应用实例。

（Vue应用实例是Vue项目工作的基础，每个Vue项目都是从创建Vue应用实例开始的。）



<> index.html X

hello-vite > <> index.html > html > head > meta

```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <link rel="icon" type="image/svg+xml" href="/vite.svg" /> 设置浏览器标签页图标
6     <meta name="viewport" content="width=device-width, initial-scale=1.0" /> 控制网页在移动设备上的显示方式
7     <title>Vite + Vue</title>
8   </head>
9   <body>
10    <div id="app"></div>
11    <script type="module" src="/src/main.js"></script>
12  </body>
13 </html>
```

JS main.js X

hello-vite > src > JS main.js

1. 导入creatApp()函数
2. 导入基础样式文件
3. 导入根组件，保存为App变量

```
const app=createApp(App)
app.mount('#app')
```

```
1 import { createApp } from 'vue'
2 import './style.css'
3 import App from './App.vue'
4
5 createApp(App).mount('#app')
```

创建Vue实例app

指定app的控制区域

hello-vite > src > App.vue > {} template > div > a

```
1 <script setup>
2 import HelloWorld from './components/HelloWorld.vue' //导入子组件
3 </script>
4
5 <template>
6   <div>
7     <a href="https://vitejs.dev" target="_blank">
8       
9     </a>
10    <a href="https://vuejs.org/" target="_blank">
11      
12    </a>
13  </div>
14  <HelloWorld msg="Vite + Vue" /> //使用子组件
15 </template>
16
17 <style scoped>
18 .logo {
19   height: 6em;
20   padding: 1.5em;
21   will-change: filter;
22   transition: filter 300ms;
23 }
24 .logo:hover {
25   filter: drop-shadow(0 0 2em #646cffaa);
26 }
27 .logo.vue:hover {
28   filter: drop-shadow(0 0 2em #42b883aa);
29 }
30 </style>
```



Vite + Vue

count is 0

Edit `components/HelloWorld.vue` to test HMR

Check out [create-vue](#), the official Vue + Vite starter

Install [Volar](#) in your IDE for a better DX

Click on the Vite and Vue logos to learn more



Vite + Vue

count is 0

Edit components/HelloWorld.vue to test HMR

Check out [create-vue](#), the official Vue + Vite starter

Install [Volar](#) in your IDE for a better DX

Click on the Vite and Vue logos to learn more

hello-vite > src > components > HelloWorld.vue

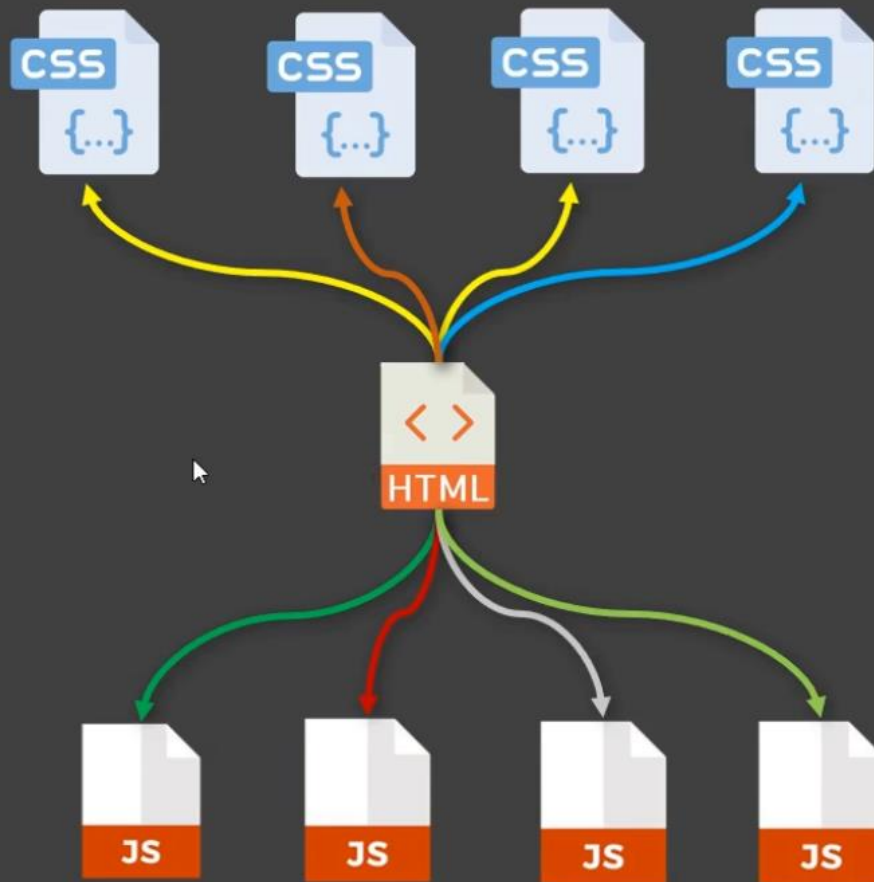
```
1  <script setup>
2  import { ref } from 'vue' //导入 ref 响应式函数
3
4  defineProps({           //定义组件 props
5    msg: String,          声明一个名为 msg 的字符串类型 prop
6  })
7
8  const count = ref(0)
9  </script>
10
11 <template>
12   <h1>{{ msg }}</h1>
13
14   <div class="card">
15     <button type="button" @click="count++">count is {{ count }}</button>
16     <p>
17       Edit
18       <code>components/HelloWorld.vue</code> to test HMR
19     </p>
20   </div>
21
22   <p>
23     Check out
24     <a href="https://vuejs.org/guide/quick-start.html#local" target="_blank">
25       >create-vue</a>
26     >, the official Vue + Vite starter
27   </p>
28   <p>
29     Install
30     <a href="https://github.com/vuejs/language-tools" target="_blank">Volar</a>
31     in your IDE for a better DX
32   </p>
33   <p class="read-the-docs">Click on the Vite and Vue logos to learn more</p>
34 </template>
```

»» 单页WEB应用

- 单页Web应用：所有的功能局限于一个Web页面中，仅在该页面的初始化时加载相应的资源（必要的HTML、CSS和JavaScript代码）。
- 在页面加载完成后，所有的操作都在这个页面上完成，且不会因用户的操作而进行页面的重新加载或跳转，而是利用JavaScript动态地变换页面的内容，从而实现页面与用户的交互。
- 优点
 - ① 良好的交互体验。
内容的改变不需要重新加载整个页面，响应速度更快。
 - ② 良好的前后端分离开发模式。
后端专注于提供API，更容易实现API的复用。
 - ③ 减轻服务器的压力。
单页Web应用中的数据是通过Ajax获取的，不需要重新加载，服务器的压力较小。

>> 组件

传统方式编写应用



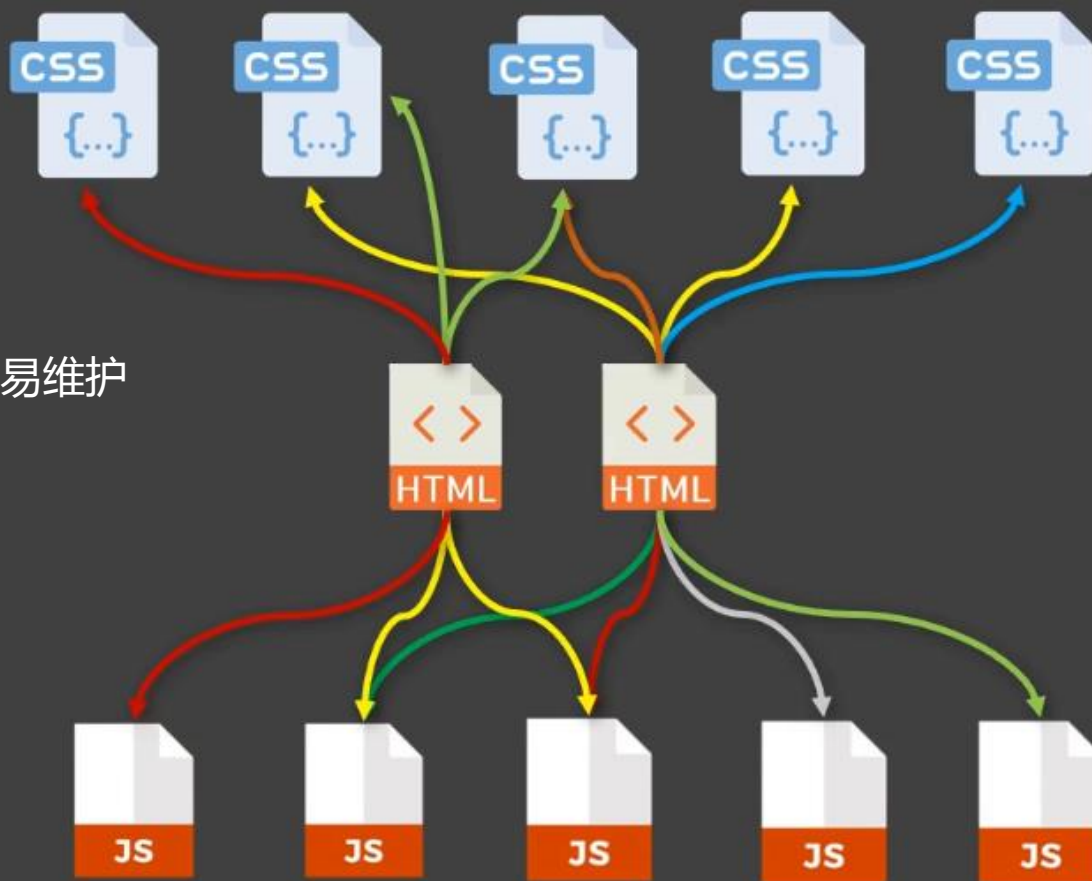
>>> 组件

传统方式编写应用



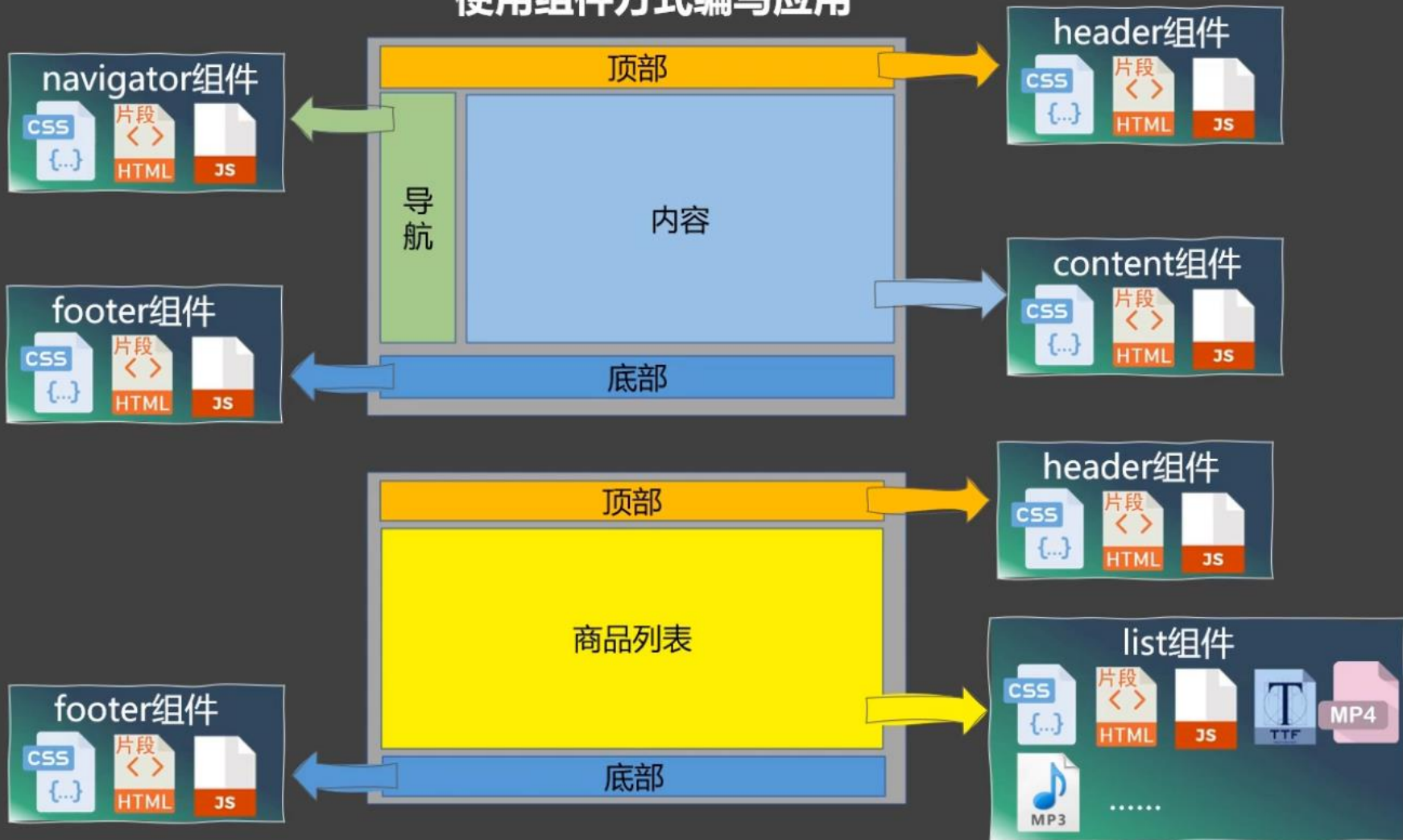
问题：

- 1、依赖关系混乱，不易维护
- 2、代码复用率不高。



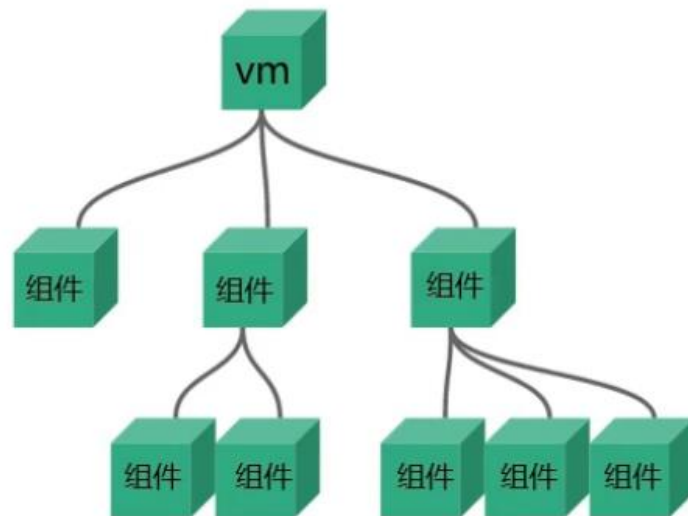
组件

使用组件方式编写应用



>> 组件

组件 —— 实现应用中**局部**功能的**代码**和**资源**的集合。



»» 7.2 单文件组件

- 每个.vue文件都可用来定义一个单文件组件。
- 每个单文件组件由模板、样式和逻辑3个部分构成。

定义一个基本的单文件组件

```
<template>
  <!-- 此处编写组件的结构 -->
</template>

<script>
/* 此处编写组件的逻辑 */
</script>

<style>
/* 此处编写组件的样式 */
</style>
```

1

模板

模板用于搭建当前组件的DOM结构，其代码写在<template>标签中。

2

逻辑

逻辑是指通过JavaScript代码处理组件的数据与业务，其代码写在<script>标签中。

3

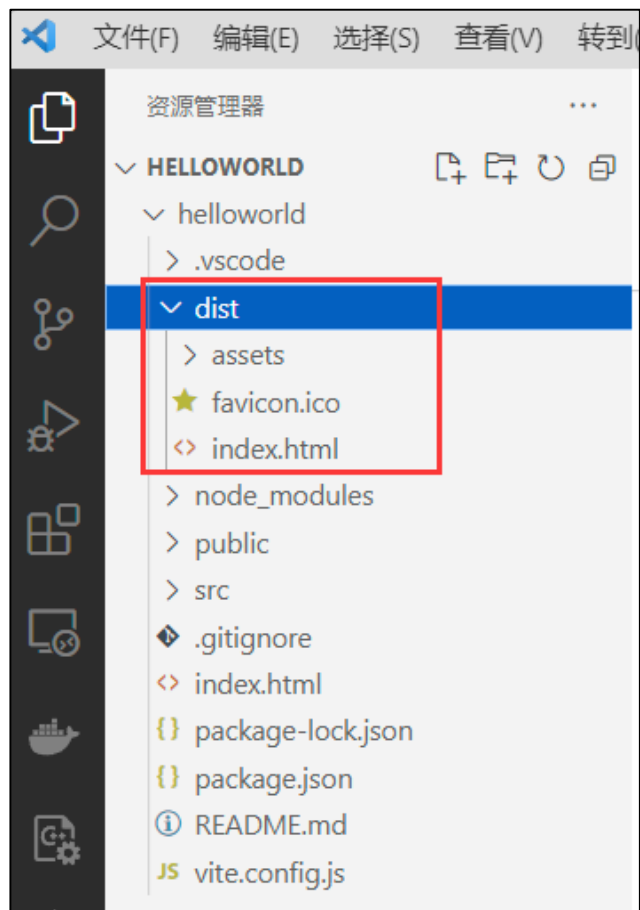
样式

样式是指通过CSS代码为当前组件设置样式，其代码写在<style>标签中。

»» 7.2 单文件组件

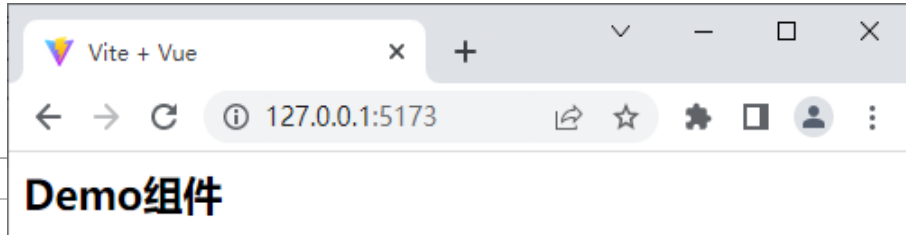
单文件组件的使用方法

- ① 使用VS Code编辑器打开
D:\code\HelloWorld\helloworld目录。
- ② 执行npm run dev命令启动服务
- ③ 将src\style.css文件中的样式代码全部删除，从而避免项目创建时自带的样式代码影响本案例的实现效果。



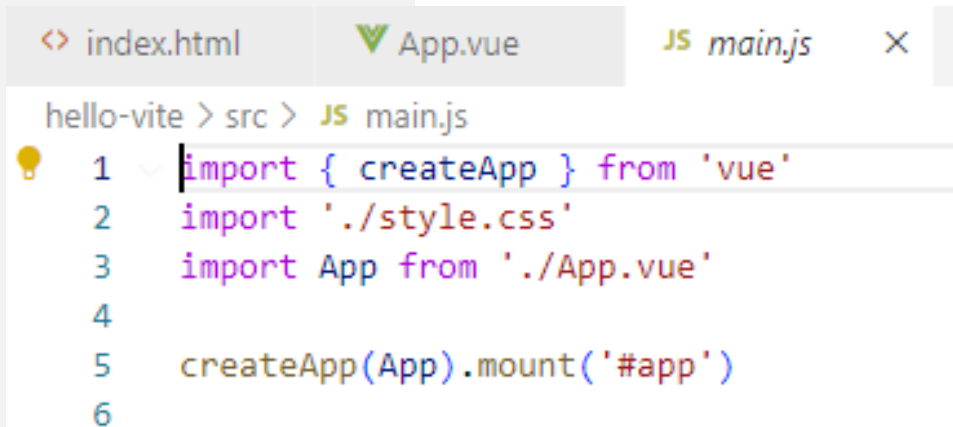
7.2 单文件组件

单文件组件的使用方法



- ④ 创建src\components\Demo.vue文件，该文件是Demo组件。

```
<template>
  <div class="demo">Demo组件</div>
</template>
<style>
.demo {
  font-size: 22px;
  font-weight: bold;
}
</style>
```



- ⑤ 修改src\main.js文件，切换页面中显示的组件。

```
import App from './components/Demo.vue'
```

»» 7.3 数据绑定

7.3.1 初识数据绑定

- Vue通过数据绑定实现了数据与页面相分离，从而实现了数据驱动视图的效果。即将页面中的数据分离出来，放到组件的<script>标签中，通过程序操作数据，并且当数据改变时，页面会自动发生改变。
- 数据绑定分为定义数据和输出数据。

定义数据

由于数据和页面是分离的，在实现数据显示之前，需要先在<script>标签中定义组件所需的数据。

输出数据

Vue为开发者提供了Mustache语法（又称为双大括号语法），将数据输出到页面中。

»» 7.3 数据绑定

7.3.1 初识数据绑定

1. 定义数据

定义数据的语法格式如下。

模块导出语法

```
<script>
export default {
  setup() {
    return {
      数据名: 数据值,
      .....
    }
  }
}
</script>
```

Vue 3提供了`setup`语法糖，简化语法，提高开发效率。

```
<script setup>
const 数据名 = 数据值
</script>
```

setup()

- `setup`函数是Vue3的新配置项
- `setup`是所有组合API“表演舞台”
- 组件中所用到的：数据、方法等均要配置在`setup`中
- `setup`函数两种返回值
 - 若返回一个对象，则对象中的属性、方法，在模板中均可以直接使用。
 - 若返回一个渲染函数，可以自定义渲染内容。（了解）
- 注意：
尽量不用和Vue2配置混用，Vue2配置（`data`，`methods`，.....）中可以访问`setup`中属性，方法。但`setup`中不能访问Vue2的配置。

» 7.3 数据绑定

7.3.1 初识数据绑定

export default : 在 Vue 单文件组件 (.vue 文件) 中，它用于导出组件的配置对象。

配置项	作用
<code>name</code>	组件名称 (用于调试和递归组件)
<code>data</code>	返回组件初始数据的函数 (必须返回对象)
<code>props</code>	声明从父组件接收的属性
<code>methods</code>	定义组件方法
<code>computed</code>	计算属性
<code>watch</code>	监听数据变化
<code>lifecycle hooks</code>	生命周期钩子 (如 <code>created</code> , <code>mounted</code>)
<code>components</code>	注册子组件

»» 7.3 数据绑定

7.3.1 初识数据绑定

2. 输出数据

- Vue使用Mustache语法时，相当于在模板中放入占位符，其语法格式如下。

`{{ 数据名 }}` 当页面渲染时，模板中的`{{ 数据名 }}`会被替换为实际的数据

- 在Mustache语法中还可以将表达式的值作为输出内容。表达式的值可以是字符串、数字等类型。

```
{{ 'Hello Vue.js' }}
```

```
{{ number + 1 }}
```

```
{{ obj.name }}
```

```
{{ ok ? 'YES' : 'NO' }}
```

三元表达式

```
{{ '<div>HTML标签</div>' }}
```

含html标签的字符串

»» 7.3 数据绑定

7.3.1 初识数据绑定

数据绑定的实现

步骤1

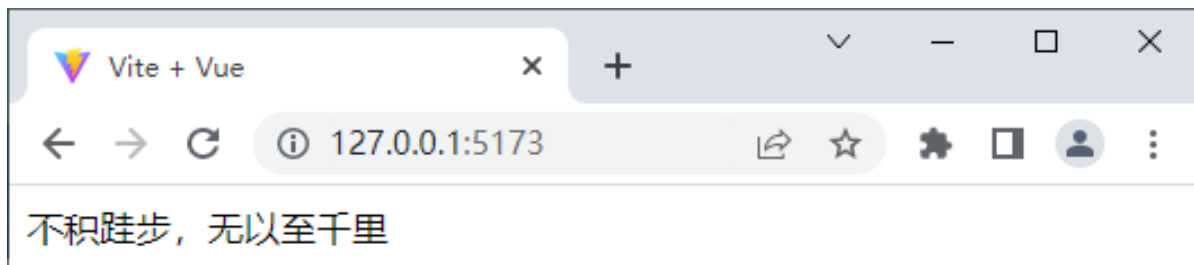
- 创建src\components\Message.vue文件。

```
<template>{{ message }}</template>  
<script setup>  
const message = '不积跬步，无以至千里'  
</script>
```

步骤2

- 修改src/main.js文件，切换页面中显示的组件。

```
import App from './components/Message.vue'
```



»» 7.3 数据绑定

7.3.1 初识数据绑定

将Vue引入HTML页面中

创建D:\vue\chapter02\helloworld.html文件，在该文件中通过<script>标签引入Vue。

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Hello World案例</title>
  <script
src="https://unpkg.com/vue@3/dist/vue.global.js"> </script>
</head>
<body>
</body>
</html>
```

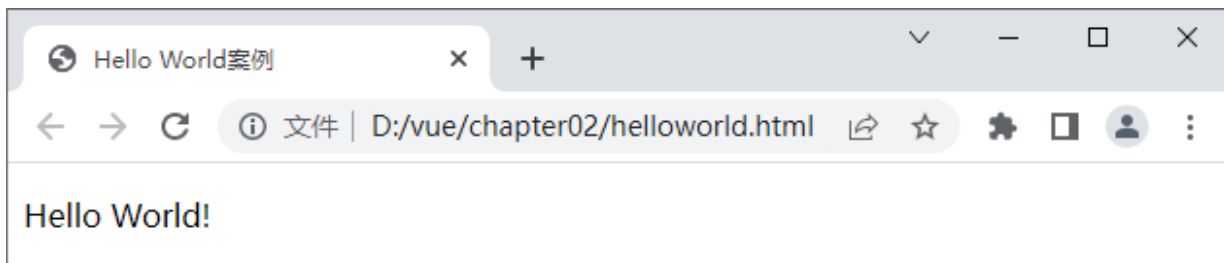
»» 7.3 数据绑定

7.3.1 初识数据绑定

如何将Vue引入HTML页面中

```
<div id="app">  
  <p>{{ message }}</p>  
</div>
```

```
<script>  
const vm = Vue.createApp({  
  setup() {  
    return {  
      message: 'Hello World!'  
    }  
  }  
})  
vm.mount('#app')  
</script>
```



» 7.3 数据绑定

7.3.2 响应式数据绑定

如果想要实现页面中数据的更新，则需要进行响应式数据绑定，也就是将数据定义成响应式数据。

Vue为开发者提供了如下函数用于定义响应式数据。

- `ref()`函数
- `reactive()`函数
- `toRef()`函数
- `toRefs()`函数

» 7.3 数据绑定

7.3.2 响应式数据绑定

1. ref()函数

`ref()`函数用于将数据转换成响应式数据，其参数为数据，返回值为转换后的响应式数据。使用`ref()`函数定义响应式数据的语法格式如下。

```
响应式数据 = ref(数据)    const xxx=ref ( initialValue )
```

如果需要更改响应式数据的值，可以使用如下语法格式进行修改。

```
响应式数据.value = 新值
```

- 创建一个包含响应式数据的引用对象，
- 在JS中操作数据：`xxx.value`
- 模板中读取数据：不需要`.value`，直接`{{xxx}}`

»» 7.3 数据绑定

7.3.2 响应式数据绑定

ref()函数的使用方法

步骤1

- 创建src\components\Ref.vue文件。

```
<template>{{ message }}</template>
<script setup>
import { ref } from 'vue'
const message = ref('会当凌绝顶，一览众山小')
setTimeout(() => {
  message.value = '锲而不舍，金石可镂'
}, 2000)
</script>
```

步骤2

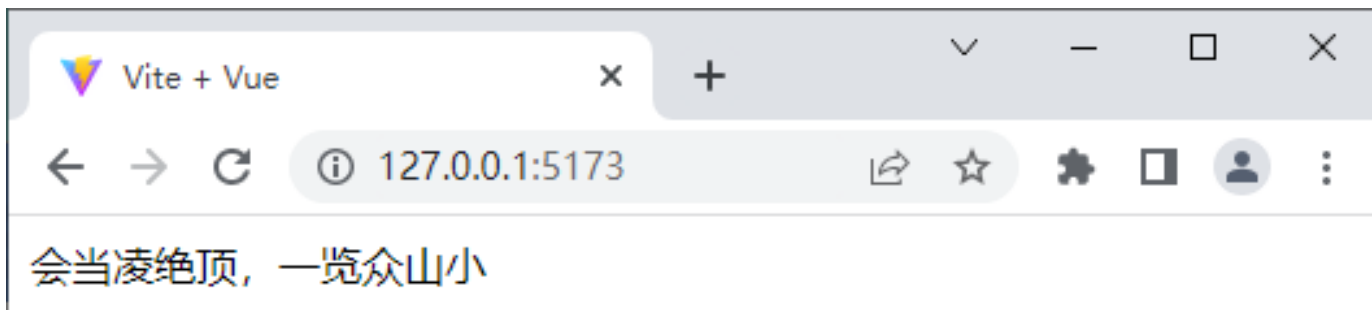
- 修改src\main.js文件，**切换**页面中**显示**的**组件**。

```
import App from './components/Ref.vue'
```

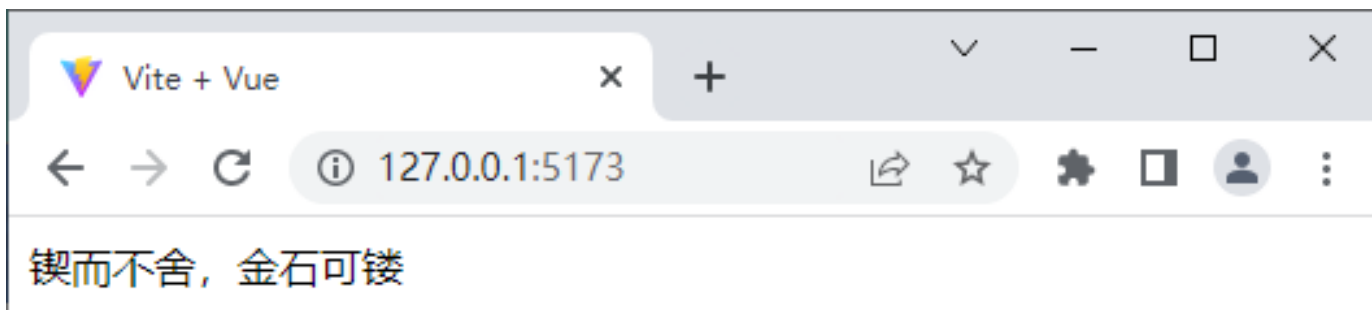
» 7.3 数据绑定

7.3.2 响应式数据绑定

在浏览器中访问<http://127.0.0.1:5173/>，初始页面效果如下。



等待2秒后的页面效果如下。



» 7.3 数据绑定

7.3.2 响应式数据绑定

2. reactive()函数

`reactive()`函数用于创建一个响应式对象或数组，将普通的对象或数组作为参数传给该函数即可。

使用`reactive()`函数定义响应式数据的语法格式如下。

响应式对象或数组 = `reactive(普通的对象或数组)`

- 语法：`const 代理对象=reactive(源对象)`，接收一个对象(数组)，返回一个代理对象。通过代理对象操作源对象内部数据。
- `reactive`定义的响应式数据是“深层次的”。

7.3 数据绑定

7.3.2 响应式数据绑定

reactive()函数的使用方法

步骤1

- 创建src\components\R

```
<template>{{ message }}</template>
<script setup>
import { ref } from 'vue'
const message = ref('会当凌绝顶，一览众山小')
setTimeout(() => {
  message.value = '锲而不舍，金石可镂'
}, 2000)
</script>
```

```
<template>{{ obj.message }}</template>
<script setup>
import { reactive } from 'vue'
const obj = reactive({ message: '不畏浮云遮望眼，自缘身在最高层' })
setTimeout(() => {
  obj.message = '欲穷千里目，更上一层楼'
}, 2000)
</script>
```

步骤2

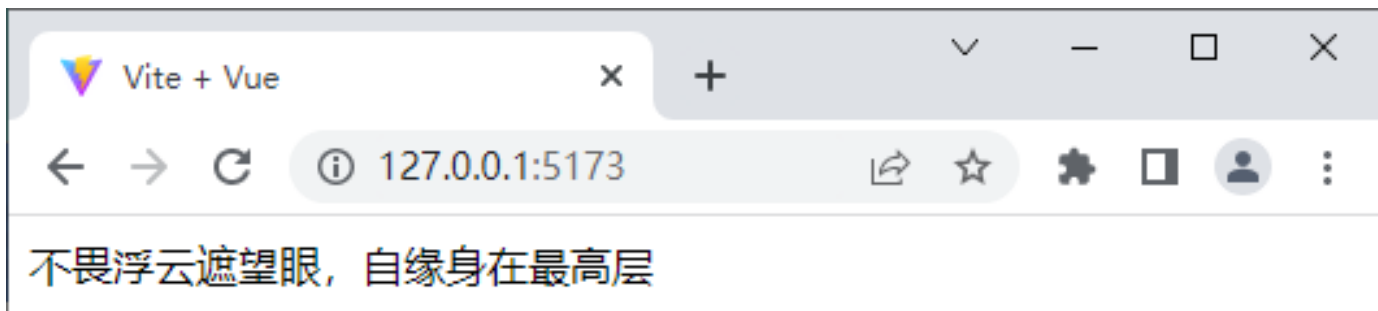
- 修改src\main.js文件，[切换](#)页面中[显示的组件](#)。

```
import App from './components/Reactive.vue'
```

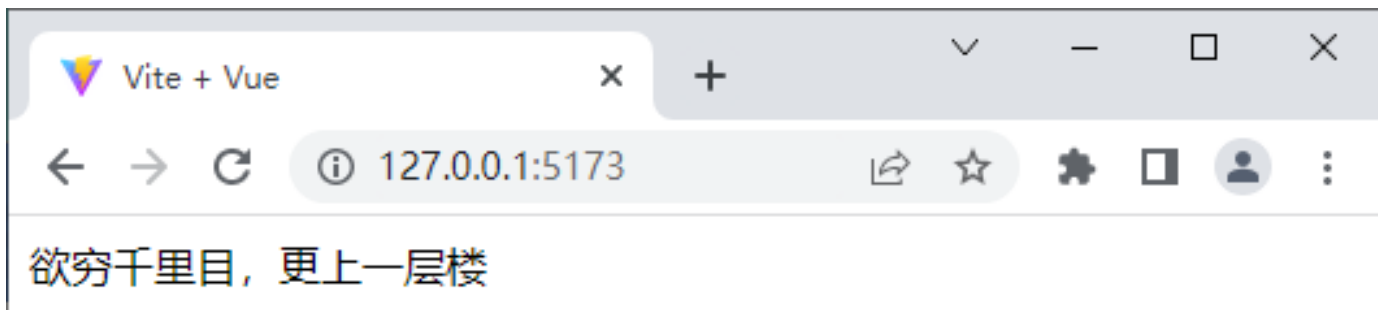
»» 7.3 数据绑定

7.3.2 响应式数据绑定

在浏览器中访问<http://127.0.0.1:5173/>，初始页面效果如下。



等待2秒后的页面效果如下图所示。



»» 7.3 数据绑定

响应式数据绑定示例

```
<script>
import {ref, reactive} from 'vue'
export default {
  name: 'App',
  setup(){
    //数据
    let name = ref('张三')
    let age = ref(18)
    let job = ref({
      type: '前端工程师',
      salary: '30K'
    })
```

```
<script>
import {ref, reactive} from 'vue'
export default {
  name: 'App',
  setup(){
    //数据
    let name = ref('张三')
    let age = ref(18)
    let job = reactive({
      type: '前端工程师',
      salary: '30K'
    })
```

```

setup(){
  //数据
  let name = ref('张三')
  let age = ref(18)
  let job = reactive({
    type: '前端工程师',
    salary: '30K',
    a: {
      b: {
        c: 666
      }
    }
  })
  let hobby = reactive(['抽烟', '喝酒', '烫头'])

  //方法
  function changeInfo(){
    name.value = '李四'
    age.value = 48
    job.type = 'UI设计师'
    job.salary = '60K'
    job.a.b.c = 999
    hobby[0] = '学习'
  }
}

```

```

<script>
import {ref, reactive} from 'vue'
export default {
  name: 'App',
  setup(){
    let person = reactive({
      name: '张三',
      age: 18,
      job: {
        type: '前端工程师',
        salary: '30K',
        a: {
          b: {
            c: 666
          }
        }
      },
      hobby: ['抽烟', '喝酒', '烫头']
    })

    //方法
    function changeInfo(){
      person.name = '李四'
      person.age = 48
      person.job.type = 'UI设计师'
      person.job.salary = '60K'
      person.job.a.b.c = 999
      person.hobby[0] = '学习'
    }
  }
}

```

»» 7.3 数据绑定

Ref和reactive对比

角度	ref	reactive
定义数据	基本数据类型	对象(或数组)数据类型
原理	Object.defineProperty()的get, set实现响应式	Proxy实现响应式, 通过reflect操作源对象内部的数据。
使用	操作数据需要.value, 读取数据时模板直接读取, 不需要.value	操作数据与读取数据, 均不需要.value

»» 7.3 数据绑定

7.3.2 响应式数据绑定

3. toRef()函数

toRef()函数用于将响应式对象中的单个属性转换为响应式数据。

使用toRef()函数定义响应式数据的语法格式如下。

响应式数据 = toRef(响应式对象, '属性名')



toRef()函数的第1个参数是响应式对象，第2个参数是待转换的属性名，返回值为转换后的响应式数据。

» 7.3 数据绑定

7.3.2 响应式数据绑定 `toRef()`函数的使用方法——示例1

步骤1

- 创建src\components\ToRef.vue文件。

```
<template>
  <div>message的值：{{ message }}</div>
  <div>obj.message的值：{{ obj.message }}</div>
</template>

<script setup>
import { reactive, toRef } from 'vue'
const obj = reactive({ message: '黑发不知勤学早，白首方悔读书迟' })
const message = toRef(obj, 'message')
setTimeout(() => {
  message.value = '少壮不努力，老大徒伤悲'
}, 2000)
</script>
```

步骤2

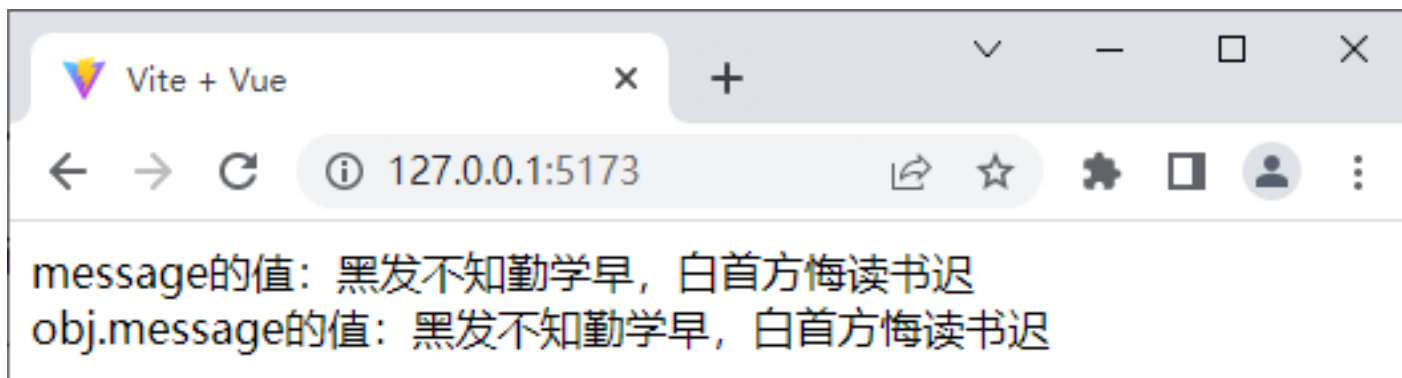
- 修改src\main.js文件，

```
import App from './components/ToRef.vue'
```

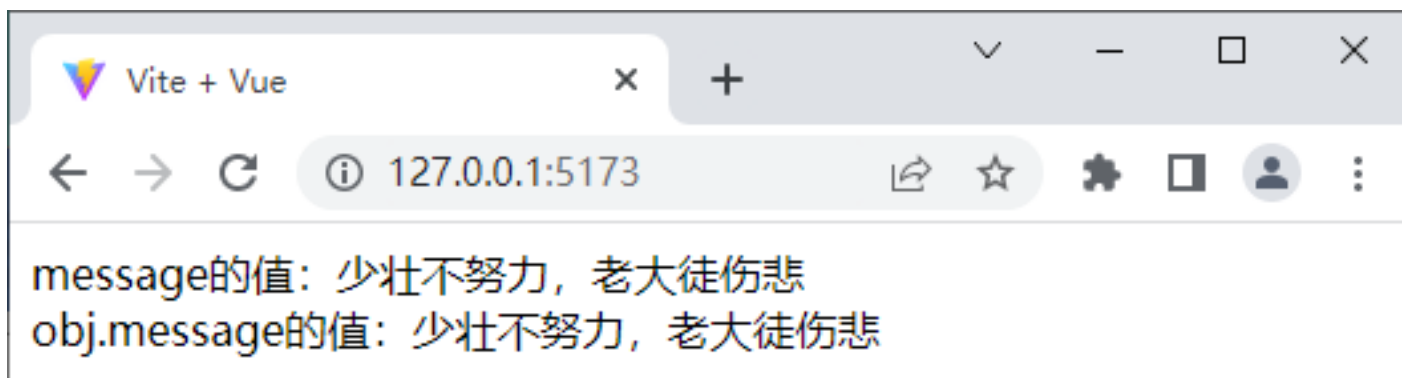
» 7.3 数据绑定

7.3.2 响应式数据绑定

在浏览器中访问<http://127.0.0.1:5173/>，初始页面效果如下。



等待2秒后的页面效果如下。



»» 7.3 数据绑定

toRef()函数的使用方法——示例2

```
import { reactive, toRef } from 'vue'
const state = reactive({
  count: 0,
  name: 'Vue3'
})
// 将 reactive 对象的属性转换为ref
const countRef = toRef(state, 'count')
// 修改源对象会同步更新
state.count++
console.log(countRef.value) // 输出 1
// 修改 ref 也会影响源对象
countRef.value++
console.log(state.count) // 输出 2
```

适用场景：

- 需要将某个属性单独传递给组合式函数
- 在模板中保持对深层属性的响应式访问

» 7.3 数据绑定

7.3.2 响应式数据绑定

4. toRefs()函数

toRefs()函数用于将响应式对象中的所有属性转换为响应式数据。
使用toRefs()函数定义响应式数据的语法格式如下。

所有属性组成的对象 = toRefs(响应式对象)

» 7.3 数据绑定

7.3.2 响应式数据绑定

toRefs()函数的使用方法

步骤1

- 创建src\components\ToRefs.vue文件。

```
<template>
  <div>message的值：{{ message }}</div>
  <div>obj.message的值：{{ obj.message }}</div>
</template>
<script setup>
import { reactive, toRefs } from 'vue'
const obj = reactive({ message: '盛年不重来，一日难再晨' })
let { message } = toRefs(obj)
setTimeout(() => {
  message.value = '及时当勉励，岁月不待人'
}, 2000)
</script>
```

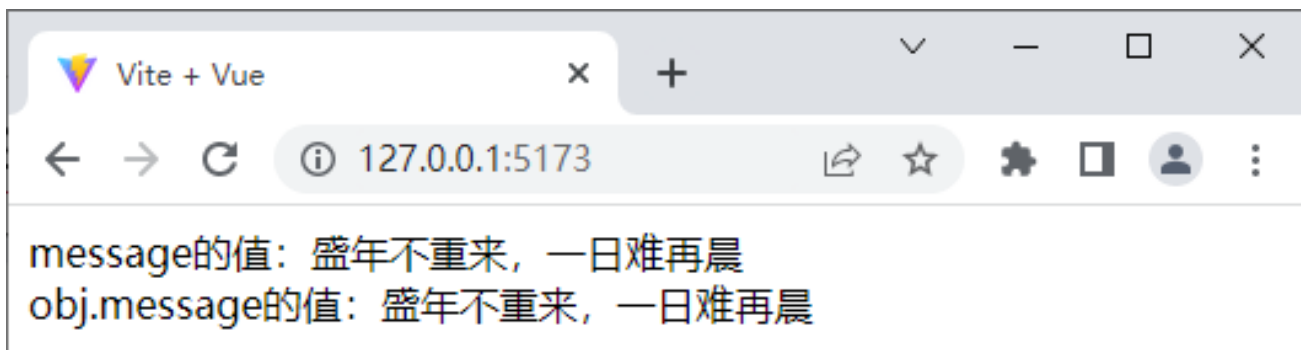
步骤2

- 修改src\main.js文件。 import App from './components/ToRefs.vue'

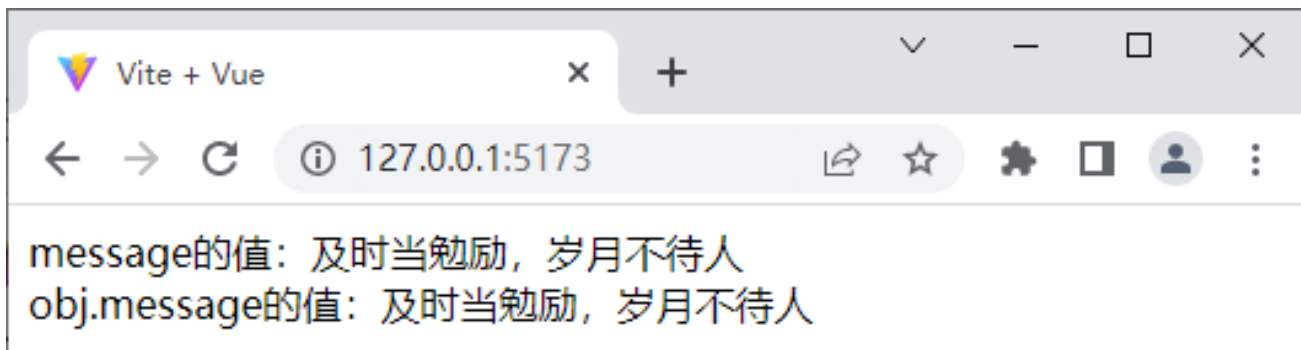
» 7.3 数据绑定

7.3.2 响应式数据绑定

在浏览器中初始页面效果如下。



等待2秒后的页面效果如下图所示。



» 7.3 数据绑定

toRefs()函数的使用方法——示例2

```
import { reactive, toRefs } from 'vue'
const state = reactive({
  count: 0,
  name: 'Vue3'
})
// 解构对象时保持响应性
const { count, name } = toRefs(state)
// 修改 ref 会影响原对象
count.value++
console.log(state.count) // 输出 1
// 在模板中直接使用 /*
<template>
  <div>{{ count }}</div>
</template> */
```

适用场景：

- 从组合式函数返回响应式对象
- 解构 props 时保持响应性
- 在模板中直接使用解构后的属性

»» 示例——props属性

```
Vite App  App.vue x App1.vue
src > App.vue > {} script setup ③
1  <script setup>
2  import Student from '@components/student.vue';
3
4  </script>
5
6  <template>
7    <div>
8      <Student/>
9    </div>
10 </template>
11
```

```
Vite App  JS main.js x App.vue
src > JS main.js > ... ②
1  import './assets/main.css'
2
3  import { createApp } from 'vue'
4  import App from './App.vue'
5  import router from './router'
6
7  const app = createApp(App)
8
9  app.use(router)
10
11 app.mount('#app')
```

```
Vite App  index.html x App.vue
<> index.html > ... ①
1  <!DOCTYPE html>
2  <html lang="">
3    <head>
4      <meta charset="UTF-8">
5      <link rel="icon" href="/favicon.ico">
6      <meta name="viewport" content="width=device-width, initial-scale=1">
7      <title>Vite App</title>
8    </head>
9    <body>
10     <div id="app"></div>
11     <script type="module" src="/src/main.js"></script>
12   </body>
13 </html>
```

»» 示例——props属性

```
Vite App  ▾ student.vue ×  ▾ App.vue
src > components > ▾ student.vue > {} template
1  <template>
2    <div>
3      <h2>学生姓名: {{ name }}</h2>
4      <h2>学生性别: {{ sex }}</h2>
5      <h2>学生年龄: {{ age }}</h2>
6    </div>
7  </template>
8
9  <script>
10   export default{
11     name: 'student',
12     data(){
13       return{
14         name: '张三',
15         sex: '男',
16         age: '18',
17       }
18     }
19   }
20 </script>
```

```
Vite App  ×  ▾ student.vue
← → ↻ http://localhost:4000/
```

学生姓名: 张三

学生性别: 男

学生年龄: 18

»» 示例——props属性

```
src > Vite App student.vue App.vue
1  <script setup>
2  import Student from '@/components/student.vue';
3
4  </script>
5
6  <template>
7    <div>
8      <Student/>
9      <hr>
10     <Student/>
11     <hr>
12     <Student/>
13   </div>
14 </template>
15
```

Vite App × student.vue App.vue
← → ↻ http://localhost:4000/

学生姓名：张三

学生性别：男

学生年龄：18

学生姓名：张三

学生性别：男

学生年龄：18

学生姓名：张三

学生性别：男

学生年龄：18

组件实例

»» 示例——props属性

Vite App

▼ student.vue ×

▼ App.vue

src > components > ▼ student.vue > {} script

```
1 <template>
2   <div>
3     <h2>学生姓名: {{ name }}</h2>
4     <h2>学生性别: {{ sex }}</h2>
5     <h2>学生年龄: {{ age }}</h2>
6   </div>
7 </template>
8
9 <script>
10
11   export default{
12     name:'student',
13     data(){
14       return{
15         msg:'我是一个工大的学生',
16       }
17     },
18     props:['name','sex','age']
19   }
20 </script>
```

Vite App

▼ student.vue

▼ App.vue ×

src > ▼ App.vue > ...

```
1 <script setup>
2   import Student from '@/components/student.vue';
3
4 </script>
5
6 <template>
7   <div>
8     <Student name="李四" sex="女" age="20"/>
9   </div>
10 </template>
```

Vite App ×

▼ student.vue

← → ↺ http://localhost:4000/

学生姓名: 李四
学生性别: 女
学生年龄: 20



我是一个工大的学生

学生姓名：李四

学生性别：女

学生年龄：20

```
<script>
  export default{
    name:'student',
    data(){
      return{
        name:'张三',
        sex:'男',
        age:'18',
      }
    }
  }
</script>
```

```
<script>
  export default{
    name:'student',
    data(){
      return{
        msg:'我是一个工大的学生',
      },
      props:['name','sex','age']
    }
  }
</script>
```





<App>

<student>

<student>

Filter State...

▼ props

name : 李四

sex : 女

age : 20

▼ data

msg : 我是一个工大的学生

▼ Routing

\$route : "/"